

Progettazione di Applicazioni Mobili

Flutter Programming

Widgets – Navigation & Dialogs

Stefano Burigat

Dipartimento di Scienze Matematiche, Informatiche e
Fisiche - Università di Udine

stefano.burigat@uniud.it



Navigation: Navigator

- Navigation widgets allow the user to move through your app or your app to move between different parts automatically (different screens, for example)
- When you start your app with a `WidgetsApp` or a `MaterialApp` you will automatically get a `Navigator` widget
 - This widget manages a set of child widgets as a stack
 - One child is visible at a time, with the rest “beneath” it
 - These children are the various screens of your apps, which are called routes in Flutter
 - The `Navigator` provides methods such as `push()` and `pop()` to add and remove routes

Navigation: Navigator example

- Consider the following two routes/screens

```
import "package:flutter/material.dart";

void main() => runApp(MyApp());

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext inContext) {
    return MaterialApp(home: FirstRoute());
  }
}

class FirstRoute extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('First Route'),
      ), // AppBar
      body: Center(
        child: ElevatedButton(
          child: Text('Open route'),
          onPressed: () {
            // Navigate to second route when
          },
        ), // RaisedButton
      ), // Center
    ); // Scaffold
  }
}
```

```
class SecondRoute extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text("Second Route"),
      ), // AppBar
      body: Center(
        child: ElevatedButton(
          onPressed: () {
            // Navigate back to first route when tapped.
          },
          child: Text('Go back!'),
        ), // RaisedButton
      ), // Center
    ); // Scaffold
  }
}
```

Navigation: Navigator example

- To switch to a new route, use the `Navigator.push()` method
- You use a `MaterialPageRoute` widget when calling `push()`, and this requires the use of a builder function
 - This is needed because when a route is navigated to, it will be built and re-built many times and that will occur in different contexts depending on when it occurs
 - The `MaterialPageRoute` is also useful because it transitions to the new route using a platform-specific animation

```
// Within the `FirstRoute` widget
onPressed: () {
  Navigator.push(
    context,
    MaterialPageRoute(builder: (context) => SecondRoute()),
  );
}
```

Navigation: Navigator example

- When you push() a new route onto a Navigator stack, it becomes visible immediately
- To go back to the previous route, you pop() it off the stack, passing the current build context

```
// Within the SecondRoute widget  
onPressed: () {  
  Navigator.pop(context);  
}
```

- Note also that a back button is automatically provided in the App bar so the pop() method is normally not necessary

Navigation: Named routes

- An alternative to the navigation approach shown in the previous example is to use named routes
 - This reduces duplicated code in the case you need to navigate to the same screen in many parts of your app
- To work with named routes, use the `Navigator.pushNamed()` function
- You also need to provide additional properties to the `MaterialApp` constructor

Navigation: Named routes example

- First create the two routes as in the previous example
- Next, provide the *initialRoute* and *routes* properties to the `MaterialApp` constructor
 - The *initialRoute* property defines which route the app should start with (by default `'/'` is the name of the first route)
 - The *routes* property defines the available named routes and the widgets to build when navigating to those routes

```
class MyApp extends StatelessWidget {  
  @override  
  Widget build(BuildContext inContext) {  
    return MaterialApp(initialRoute: '/', routes: {  
      // When navigating to the "/" route, build the FirstScreen widget.  
      '/': (context) => FirstRoute(),  
      // When navigating to the "/second" route, build the SecondScreen widget.  
      '/second': (context) => SecondRoute(),  
    }); // MaterialApp  
  }  
}
```

Navigation: Named routes example

- With the widgets and routes in place, trigger navigation by using the `Navigator.pushNamed()` method

```
// Within the `FirstScreen` widget
onPressed: () {
  // Navigate to the second screen using a named route.
  Navigator.pushNamed(context, '/second');
}
```

- To navigate back to the first screen, use the `Navigator.pop()` function

```
// Within the SecondRoute widget
onPressed: () {
  Navigator.pop(context);
}
```


Navigation: BottomNavigationBar

- The BottomNavigationBar widget provides a bar at the bottom of the screen with icons and/or text that the user can click to move between parts of your app

```
import "package:flutter/material.dart";

void main() => runApp(MyApp());

class MyApp extends StatefulWidget {
  MyApp({Key key}) : super(key: key);

  @override
  _MyApp createState() => _MyApp();
}

class _MyApp extends State {
  var _currentPage = 0;
  var _pages = [
    Text("Page 1 - Announcements"),
    Text("Page 2 - Birthdays"),
    Text("Page 3 - Data")
  ];
}
```

```
@override
Widget build(BuildContext context) {
  return MaterialApp(
    title: "Flutter Playground",
    home: Scaffold(
      body: Center(child: _pages.elementAt(_currentPage)),
      bottomNavigationBar: BottomNavigationBar(
        items: [
          BottomNavigationBarItem(
            icon: Icon(Icons.announcement),
            title: Text("Announcements")), // BottomNavigationBarItem
          BottomNavigationBarItem(
            icon: Icon(Icons.cake), title: Text("Birthdays")), // Bott
          BottomNavigationBarItem(
            icon: Icon(Icons.cloud), title: Text("Data")), // BottomNa
        ],
        currentIndex: _currentPage,
        fixedColor: Colors.red,
        onTap: (int inIndex) {
          setState(() {
            _currentPage = inIndex;
          });
        },
      )); // BottomNavigationBar, Scaffold, MaterialApp
}
```

Navigation: BottomNavigationBar

- Here, we start by creating a stateful widget
 - This is necessary because the toplevel widget is built once unless it has state that changes, which is precisely what we need to happen when the user clicks one of the items on the bar
- To deal with state, we need to create a class extending from StatefulWidget and one extending from State
 - the class that actually is your widget is the one extending from State, not the one extending from StatefulWidget

Navigation: BottomNavigationBar

- The `bottomNavigationBar` property of the `Scaffold` widget takes as its value an instance of `BottomNavigationBar`, which has an `items` property
 - This property is a list of `BottomNavigationBarItem` widgets
 - Each of those can have an icon and a title as we seem fit
 - The `currentIndex` property of the `BottomNavigationBar` tells us which of the items on the bar is currently selected
 - The `fixedColor` property determines what color to make that selected item
- The `onTap` function is passed the index of the tapped item and the `setState` function triggers Flutter to rebuild the widget which will update the screen

Navigation: NavigationBar

- The NavigationBar is the Material Design 3 equivalent to the BottomNavigationBar
- To transform a BottomNavigationBar into a NavigationBar you need to:
 - Change the bottomNavigationBar property from BottomNavigationBar() to NavigationBar()
 - Replace the property “items” with the property “destinations”
 - Replace the property “currentIndex” with the property “selectedIndex”
 - Replace the property “onTap” with the property “onDestinationSelected”
 - Use the styling properties supported by NavigationBar

Navigation: TabBar and TabBarView

- A TabBarView is essentially a stack of screens where only one is visible at a time, and the user can move between them
- The way a screen becomes visible is by user interaction with a TabBar, clicking the icon for one of the tabs or swiping between them

```
import "package:flutter/material.dart";

void main() => runApp(MyApp());

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: DefaultTabController(
        length: 3,
        child: Scaffold(
          appBar: AppBar(
            title: Text("Flutter Playground"),
            bottom: TabBar(tabs: [
              Tab(icon: Icon(Icons.announcement)),
              Tab(icon: Icon(Icons.cake)),
              Tab(icon: Icon(Icons.cloud))
            ]), // TabBar, AppBar
          body: TabBarView(children: [
            Center(child: Text("Announcements")),
            Center(child: Text("Birthdays")),
            Center(child: Text("Data"))
          ])))); // TabBarView, Scaffold, DefaultTabCo
  }
}
```

Navigation: TabBar and TabBarView

- Behind the scenes, a TabController widget will be responsible for keeping track of what tab is current and the content of each
- Most of the time you'll just use the DefaultTabController widget as the value of the home property of a MaterialApp widget, which takes care of all the details for you
 - you have to tell the TabController how many tabs there are via the length property
 - An array of Tabs widgets is used for the tabs property of the TabBar
 - The content for each tab is set in the children property of the TabBarView and can be a complex widget tree

Navigation: Stepper

- The Stepper widget is used to walk the user through a defined sequence of events

```
import "package:flutter/material.dart";

void main() => runApp(MyApp());

class MyApp extends StatefulWidget {
  MyApp({Key key}) : super(key: key);

  @override
  _MyApp createState() => _MyApp();
}

class _MyApp extends State {
  var _currentStep = 0;

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: "Flutter Playground",
      home: Scaffold(
        appBar: AppBar(title: Text("Flutter Playground")),
        body: Stepper(
          type: StepperType.vertical,
          currentStep: _currentStep,
          onStepContinue: _currentStep < 2
            ? () => setState(() => _currentStep += 1)
            : null,
          onStepCancel: _currentStep > 0
            ? () => setState(() => _currentStep -= 1)
            : null,
          steps: [
            Step(
              title: Text("Get Ready"),
              isActive: true,
              content: Text("Let's begin..."), // Step
            ),
            Step(
              title: Text("Get Set"),
              isActive: true,
              content: Text("Ok, just a little more..."),
            ),
            Step(
              title: Text("Go!"),
              isActive: true,
              content: Text("And, we're done!") // Step
            )
          ]
        ); // Stepper, Scaffold, MaterialApp
      )
    );
  }
}
```

Navigation: Stepper

- You first need to tell the the Stepper widget whether you want your steps to be displayed vertically or horizontally via the type property
- You also need to tell it what step the user is currently on, and that's done here by virtue of the `_currentStep` variable
 - This is a stateful widget since the value of that variable is what determines what step is displayed
- We also must provide some code to the Stepper for when the user clicks the Continue and Cancel buttons
- Next, we need to define the steps of the sequence; each one with a Step widget

Dialogs&co: Tooltip

- The Tooltip widget is handy for showing a description of some other widget when you perform some appropriate action (most commonly long-pressing a button)
- To apply one, you wrap the target widget in a Tooltip

```
Tooltip(  
  message: "Tap me at your own risk!",  
  child: TextButton(  
    onPressed: () => print("BOOM!"),  
    child: Text("Do not tap!"),  
  ), // TextButton  
) // Tooltip
```

- Some widgets have a tooltip property that automatically wraps the widget in a Tooltip, but you can do so manually if not

Dialogs&co: SimpleDialog

- A SimpleDialog is a popup element that offers the user a choice between several options
 - Most of the time, the options are rendered using the SimpleDialogOption widget
 - The SimpleDialog can optionally have some title text which is displayed above the options
- An instance of SimpleDialog is normally passed to the showDialog() function for display

Dialogs&co: SimpleDialog

```
import "package:flutter/material.dart";

void main() => runApp(MyApp());

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(home: Scaffold(body: Home()));
  }
}

class Home extends StatelessWidget {
  @override
  Widget build(BuildContext inContext) {
    Future _showIt() async {
      switch (await showDialog(
        context: inContext,
        builder: (BuildContext inContext) {
          return SimpleDialog(
            title: Text("What's your favorite food?"),
            children: [
              SimpleDialogOption(
                onPressed: () {
                  Navigator.pop(inContext, "broccoli");
                },
                child: Text("Broccoli"), // SimpleDialogOption
              ),
              SimpleDialogOption(
                onPressed: () {
                  Navigator.pop(inContext, "steak");
                },
                child: Text("Steak")) // SimpleDialogOption
              ],
            )); // SimpleDialog
    }
  }

  case "broccoli":
    print("Broccoli");
    break;
  case "steak":
    print("Steak");
    break;
}

return Scaffold(
  body: Center(
    child: ElevatedButton(
      child: Text("Show it"),
      onPressed: _showIt)); // RaisedButton, Center
  )
);
```

Dialogs&co: SimpleDialog

- When the RaisedButton is tapped, it calls the `_showIt()` function
- This function awaits the return value from `showDialog()` as the value of the switch statement
- When the user then clicks one of the options, first the dialog must be hidden, which is what the `Navigator.pop()` call does
- The second argument to `pop()` is the value to return, which the two case statements then handle to `print()` the result to the console

Dialogs&co: AlertDialog

- The AlertDialog is much like the SimpleDialog except that it is meant for urgent situations that require their immediate attention and typically don't require more than a binary choice of some sort (or no choice at all)

- Building on the previous code, all we need to change is the `_showIt()` function

Il return serve solo se vogliamo recuperare il valore tornato da `pop()`

```
_showIt() {  
  return showDialog(  
    context: inContext,  
    barrierDismissible: false,  
    builder: (BuildContext context) {  
      return AlertDialog(  
        title: Text("We come in peace..."),  
        content: Center(  
          child:  
            Text("...shoot to kill shoot to kill shoot to kill"),  
          actions: [  
            TextButton(  
              child: Text("Beam me up, Scotty!"),  
              onPressed: () {  
                Navigator.of(context).pop();  
              }) // FlatButton  
            ]); // AlertDialog  
    });  
  }  
}
```

Dialogs&co: SnackBar

- A SnackBar is a lightweight message component that shows a transient message at the bottom of the screen for some period of time, and optionally with a single action
- We have to use the ScaffoldMessenger.of(context) call to get a reference to the ScaffoldMessengerState that has a showSnackBar() method, which is what we call

```
void showSnackBar() {  
    ScaffoldMessenger.of(context).showSnackBar(  
        SnackBar(  
            backgroundColor: Colors.grey[10],  
            content: Text("Button pressed"),  
            duration: Duration(seconds: 4),  
            action: SnackBarAction(  
                label: "OK",  
                onPressed: (){  
                    print("OK pressed");  
                },  
            ), // SnackBarAction  
        ), // SnackBar  
    );  
}
```

Dialogs&co: BottomSheets

- Bottom sheets, as provided by the BottomSheet widget are widgets displayed at the bottom of the screen to show additional content to the user and/or ask them for a choice
- Here we use `showModalBottomSheet()` to show a modal BottomSheet
 - What content you show on it is up to you

```
_showIt() {  
  showModalBottomSheet(  
    context: inContext,  
    builder: (BuildContext inContext) {  
      return new Column(mainAxisSize: MainAxisSize.min, children: [  
        Text("What's your favorite pet?"),  
        TextButton(  
          child: Text("Dog"),  
          onPressed: () {  
            Navigator.of(inContext).pop();  
          },  
        ), // FlatButton  
        TextButton(  
          child: Text("Cat"),  
          onPressed: () {  
            Navigator.of(inContext).pop();  
          },  
        ), // FlatButton  
        TextButton(  
          child: Text("Ferret"),  
          onPressed: () {  
            Navigator.of(inContext).pop();  
          }) // FlatButton  
      ]); // Column  
    });  
}
```